

GRENOBLE-INP ENSIMAG

INGÉNIERIE DES SYSTÈMES
D'INFORMATION

2A ISI

GRENOBLE-INP ENSIMAG

INFORMATION SYSTEMS
ENGINEERING

2A ISI



**ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET DE MATHÉMATIQUES
APPLIQUÉES DE GRENOBLE**

PROJET GoMile

Cahier des charges

Réalisé par :

DUONG DANG KHOA DANG
DIAKITE ALPHA OUSMANE
JOUNI YOUSSEF
MAOUDE SAMIR
OUDRHIRI IDRISSE SAFWANE

CLIENT :

GoMile Entreprise

2026–2027

Table des matières

1	Introduction	3
1.1	Contexte du Projet	3
1.2	Problématique	3
1.3	Présentation de la Solution	4
1.4	Objectifs et Portée	4
2	Objectifs du projet	5
2.1	Objectifs Stratégiques	5
2.2	Objectifs Fonctionnels	5
2.3	Objectifs Techniques	5
3	Cas d'Usage et Modélisation des Interactions	7
3.1	Définition des Acteurs	7
3.2	Inventaire des Cas d'Usage	7
3.3	Diagramme de Séquence : Le "Golden Path"	8
4	Conception de l'Interface Web et Logique Applicative	9
4.1	Choix Technologiques (La Stack)	9
4.2	Modélisation des États de l'Interface	9
4.3	Architecture des Dashboards	9
4.4	Flux de données en Temps Réel (Séquence UI)	10
4.5	Directives Techniques : Langages et Frameworks (Stack Front-End)	11
5	Architecture Backend & Conception de l'API	13
5.1	Directives Techniques : La Stack Backend	13
5.2	Diagramme d'Architecture et Flux de Données	14
5.3	Modélisation de la Base de Données (MCD Simplifié)	14
5.4	Directives Opérationnelles pour les Développeurs Backend	14
6	Extension WooCommerce (Plugin Commerçant)	16
6.1	Fonctionnalités attendues	16
6.2	Contraintes techniques	16
6.3	Flux principal attendu	17
6.4	Livrables attendus	18

7	Application Mobile Livreur	19
7.1	Directives Techniques : La Stack Mobile	19
7.2	Diagramme d'États : Cycle d'Activité et Tracking GPS	19
7.3	Directives d'Implémentation par Fonctionnalité	19
8	Conclusion	22



1 Introduction

1.1 Contexte du Projet

Dans un paysage économique marqué par la transition numérique accélérée, le commerce de proximité fait face à un défi majeur : la **logistique du dernier kilomètre**. Si les grandes plateformes e-commerce disposent d'infrastructures massives, les commerçants indépendants (restaurants, fleuristes, boutiques de prêt-à-porter) peinent à proposer une expérience de livraison rapide, fiable et automatisée sans sacrifier leurs marges ou leur indépendance.

Le présent projet vise à combler ce fossé technologique en offrant une solution de gestion de livraison *end-to-end*. Il ne s'agit pas simplement d'une application de mise en relation, mais d'un écosystème complet intégrant la gestion des stocks, le dispatch intelligent et le suivi sécurisé.

1.2 Problématique

Le marché actuel de la livraison est paradoxal : alors que la demande pour le "cliquer-récupérer" et la livraison instantanée n'a jamais été aussi forte, les solutions existantes créent des barrières infranchissables pour les petits acteurs et les nouveaux entrepreneurs du web. Notre analyse identifie trois problèmes majeurs :

- **La rupture de flux et synchronisation** : Actuellement, un commerçant doit souvent gérer deux interfaces distinctes : son catalogue de vente (WooCommerce) et son interface de livraison (Uber, Stuart, ou services postaux). Cette absence d'interopérabilité force une saisie manuelle des données client, augmentant drastiquement le risque d'erreurs d'adressage et le temps de traitement par commande.
- **La complexité d'accès pour les nouveaux acteurs** : Les solutions de livraison "à la demande" sont souvent conçues pour de grandes enseignes avec des équipes techniques dédiées. Pour un **vendeur indépendant ou un influenceur** lançant sa propre boutique, l'intégration d'un service comme Uber Direct est techniquement complexe et contractuellement rigide. Ces acteurs se retrouvent alors contraints d'utiliser des services postaux traditionnels (type La Poste), inadaptés à la livraison locale immédiate et n'offrant pas de suivi dynamique.
- **Les barrières financières et techniques au développement** : Développer un module de livraison personnalisé au sein d'une boutique en ligne représente un coût de développement et de maintenance prohibitif pour une PME. En l'absence d'une solution "Plug-and-Play" abordable, beaucoup renoncent à proposer la livraison, perdant ainsi une part de marché considérable face aux géants du secteur.

- **L’absence de suivi en temps réel pour le client final** : La frustration du client provient souvent de l’opacité du trajet du colis. Le manque d’outils simples permettant de visualiser le livreur sur une carte, sans obliger le client à installer une application mobile supplémentaire, dégrade l’expérience d’achat globale.

1.3 Présentation de la Solution

Le projet repose sur une architecture modulaire composée de quatre piliers technologiques :

1. **Le Plugin (Focus initial : WooCommerce)** : Une extension installée directement sur la boutique du commerçant qui automatise la transmission des commandes à notre infrastructure.
2. **L’Espace Web (SaaS)** : Un tableau de bord centralisé permettant aux enseignes de gérer leurs locaux, de consulter leurs historiques, de gérer leurs abonnements et de soumettre leurs documents de conformité.
3. **L’Application Mobile Livreur** : Un outil de terrain dédié à la réception des missions, à la navigation GPS et à la sécurisation des transactions via un système de double validation par code (Handshake).
4. **L’API Centrale** : Le cerveau du système qui orchestre l’algorithme de dispatching, la gestion de la base de données géographique et la communication inter-modules.

1.4 Objectifs et Portée

L’objectif final est de fournir une infrastructure scalable. Bien que le lancement se concentre sur l’écosystème WordPress/WooCommerce, la conception de l’API permet une extension rapide vers Shopify ou des systèmes de caisse physiques (POS). Le projet garantit une transparence totale grâce à un système de notation bidirectionnel et une gestion de portefeuille (Wallet) permettant aux livreurs une flexibilité totale dans la perception de leurs revenus.

En résumé, notre solution transforme la livraison locale en un service industriel, sécurisé et accessible, replaçant le commerçant au centre de sa relation client.

2 Objectifs du projet

L'objectif principal de ce projet est de concevoir une infrastructure technologique robuste capable d'orchestrer en temps réel le flux complexe de la livraison urbaine. Cette section détaille les ambitions stratégiques, fonctionnelles et techniques du système.

2.1 Objectifs Stratégiques

- **Démocratisation de la logistique premium** : Permettre à des acteurs de petite taille (indépendants, artisans, influenceurs) d'accéder à une puissance logistique équivalente aux géants du secteur via une installation simplifiée ("Plug-and-Play").
- **Indépendance des commerçants** : Offrir une alternative directe aux places de marché centralisées en permettant aux commerçants de conserver la propriété de leur relation client et de leurs données.
- **Valorisation du métier de livreur** : Créer un écosystème de travail transparent où la qualité de service est récompensée par un système de notation équitable et une gestion financière autonome (Wallet).

2.2 Objectifs Fonctionnels

- **Zéro saisie manuelle (Frictionless)** : L'automatisation doit être totale. Dès qu'une commande est validée sur le CMS (WooCommerce), le processus de recherche de livreur doit se déclencher sans intervention humaine.
- **Sécurisation du transfert de responsabilité** : Garantir l'intégrité de la chaîne de livraison via un protocole de "Double Handshake" (codes OTP générés par l'API) à chaque étape critique (Enlèvement et Remise).
- **Gestion Multi-Entités** : Permettre à une seule enseigne de piloter dynamiquement plusieurs points de vente physiques, chacun associé à son propre stock et ses propres clés d'identification API.
- **Traçabilité totale** : Fournir un historique immuable des opérations, incluant le suivi géographique, les délais de traitement et les flux financiers.

2.3 Objectifs Techniques

- **Performance du Dispatching** : L'algorithme de matching doit être capable de traiter les requêtes de proximité en quelques secondes pour garantir une réactivité optimale.
- **Haute Disponibilité** : Assurer un taux de disponibilité du service (Uptime) de 99.9%, le suivi client par SMS étant une fonction critique pour l'image de marque

du commerçant.

- **Conformité et Sécurité** : Automatisation des processus de vérification (KYC/KYB) et respect strict des directives RGPD concernant les données de géo-localisation et les informations personnelles des clients.

Architecture du Flux Sécurisé

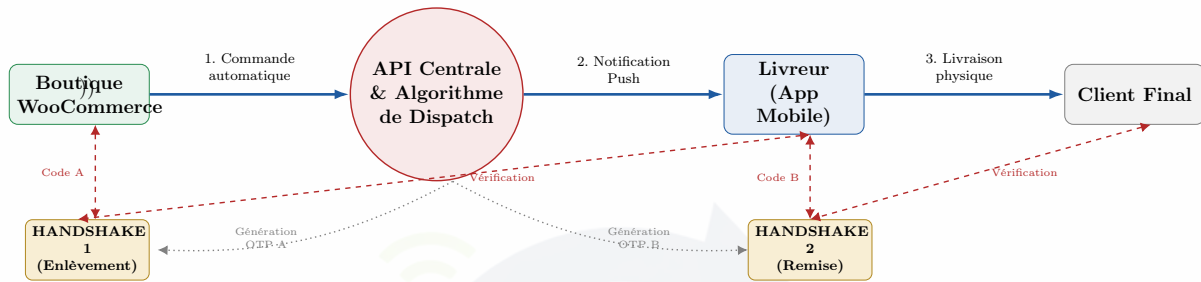


FIGURE 1 – Schéma des objectifs : Automatisation et sécurisation des échanges.

3 Cas d'Usage et Modélisation des Interactions

Cette section détaille les fonctionnalités du système à travers les interactions entre les différents acteurs identifiés. L'objectif est de définir précisément le "qui fait quoi" afin de structurer la logique métier et les permissions d'accès.

3.1 Définition des Acteurs

Le système repose sur l'interaction de quatre acteurs principaux :

- **Commerçant / Enseigne (Donneur d'ordre)** : Entité qui utilise le système (via API ou Interface) pour déléguer la livraison de colis. Il est le point d'origine de la commande.
- **Livreur (Prestataire)** : Utilisateur mobile indépendant ou salarié chargé de la récupération et du dépôt physique du colis.
- **Client Final (Destinataire)** : Bien qu'il n'interagisse pas directement avec l'administration du système, il est l'acteur clé de la validation finale (système de handshake).
- **Administrateur** : Responsable de la maintenance opérationnelle, de la validation des profils (KYC) et de la résolution des litiges.

3.2 Inventaire des Cas d'Usage

Les fonctionnalités sont segmentées par acteur pour garantir une séparation claire des responsabilités.

Pour le Commerçant

- **Gestion du compte** : Inscription, configuration des points de collecte (locaux).
- **Intégration technique** : Génération et révocation de clés API pour l'automatisation.
- **Suivi opérationnel** : Consultation du statut des courses en temps réel et accès à l'historique des facturations.

Pour le Livreur

- **Onboarding** : Création de profil et soumission des documents légaux (KYC).
- **Gestion d'activité** : Passage en mode "En ligne" pour recevoir des notifications de missions.
- **Exécution** : Acceptation d'une course, navigation, et validation des étapes par code (Handshake).

Pour l'Administrateur

- **Modération** : Validation ou rejet des documents des livreurs.
- **Supervision** : Monitoring des flux et intervention en cas de litige entre un commerçant et un livreur.

3.3 Diagramme de Séquence : Le "Golden Path"

Le diagramme suivant illustre le flux nominal (scénario idéal) d'une livraison, depuis le déclenchement par le plugin e-commerce jusqu'à la clôture de la mission. C'est ici que l'on visualise l'importance du double code de validation (Handshake).

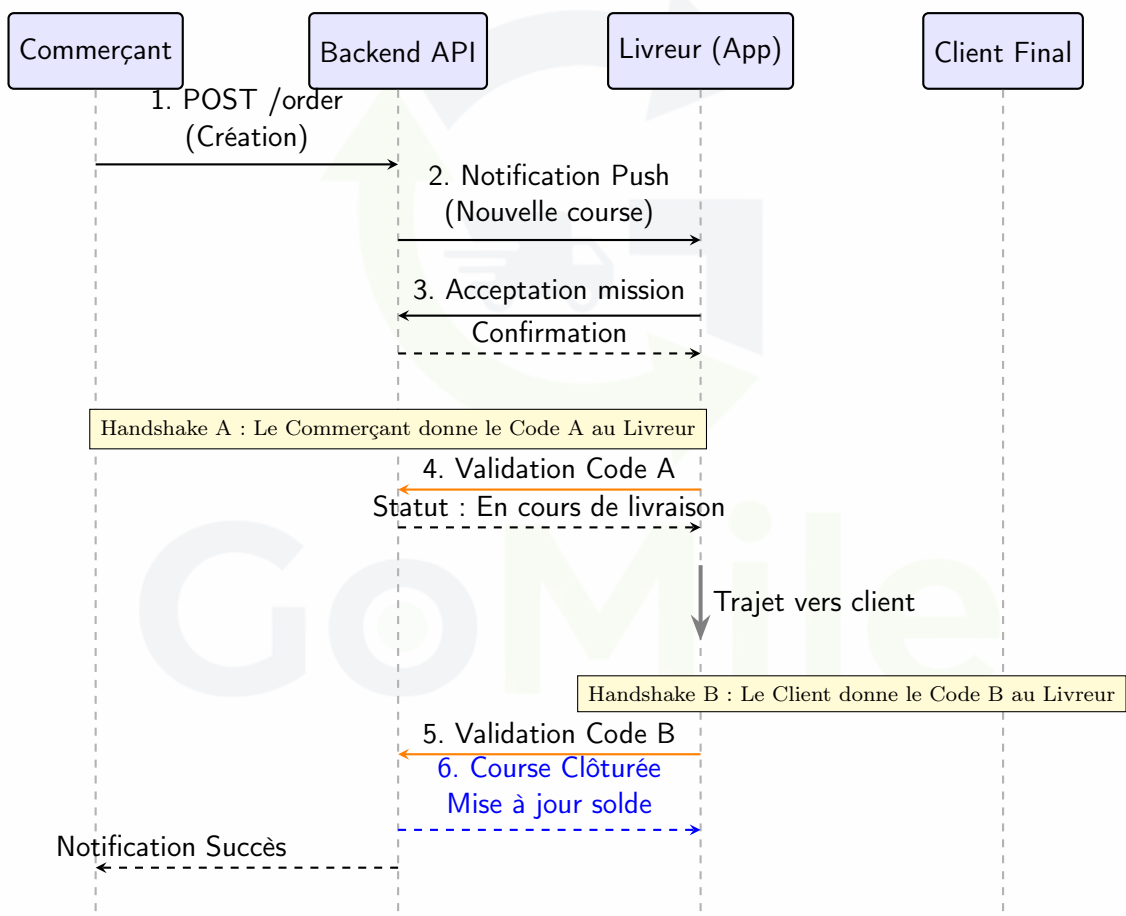


FIGURE 2 – Diagramme de séquence du flux nominal (Golden Path) avec double validation par codes.

4 Conception de l'Interface Web et Logique Applicative

L'interface Web constitue le poste de contrôle principal pour les commerçants et l'outil de régulation pour les administrateurs. Cette section détaille les choix technologiques, la modélisation des états et les directives de conception pour garantir une plateforme réactive et maintenable.

4.1 Choix Technologiques (La Stack)

Il est impératif d'utiliser une stack moderne qui favorise la réactivité (temps réel) et la maintenabilité du code.

@ 11 X @	Composant	Technologie	Justification
Framework	Next.js 14+	(App Router)	Rendu hybride (SSR/Client), routing robuste et optimisation SEO du site vitrine.
Langage	TypeScript		Typage strict pour sécuriser la manipulation des objets complexes (Commande, Utilisateur).
Styling	Tailwind CSS + Shadcn/UI		Design système cohérent, implémentation rapide et accessibilité (normes ARIA) native.
Gestion d'état	TanStack Query		Gestion ultra-performante du cache serveur et des synchronisations avec l'API.
Temps Réel	WebSockets (Socket.io)		Crucial pour le suivi GPS du livreur en direct sans rafraîchissement de la page.

TABLE 1 – Stack technologique de l'interface Web

4.2 Modélisation des États de l'Interface

L'interface utilisateur est directement pilotée par l'état (statut) de la commande. Le diagramme d'états ci-dessous représente le cycle de vie d'une vue de commande côté Commerçant.

4.3 Architecture des Dashboards

L'application Web se divise en trois grandes zones isolées et sécurisées selon le rôle de l'utilisateur :

Dashboard Commerçant (La Tour de Contrôle)

- **Vue Carte** : Intégration d'une cartographie (ex : Google Maps, ...) pour géolocaliser les livreurs affectés en temps réel.

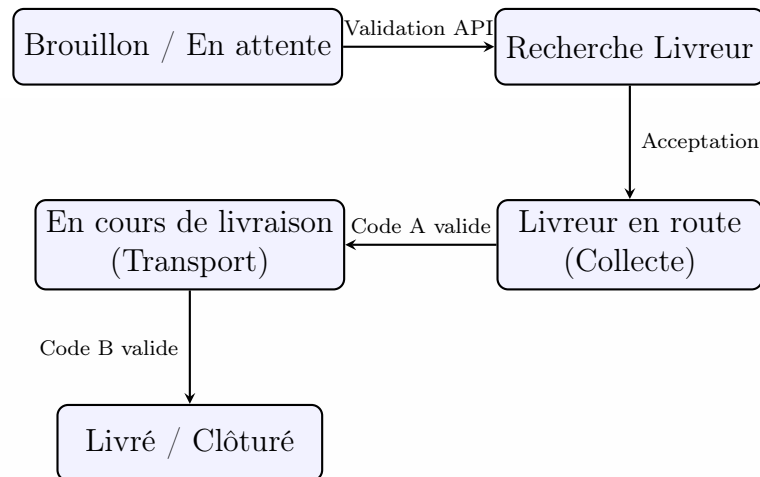


FIGURE 3 – Diagramme d'états du cycle de vie d'une commande (Vue Commerçant).

- **Gestion API** : Interface dédiée pour générer/révoquer des locaux ou boutiques physiques.
- **Validation Handshake** : Interface d'affichage bien visible du **Code A** à transmettre au livreur lors de son arrivée au local.

Panel Administrateur (Le Back-Office)

- **Files d'attente KYC** : Interface de revue des pièces d'identité des livreurs avec outils de zoom, validation ou rejet avec motif.
- **Arbitrage et Monitoring** : Console d'intervention permettant de forcer l'annulation ou la validation d'une course bloquée en cas de litige.

Interface Web Livreur (Responsive)

- Bien que l'application mobile native soit l'outil principal du livreur sur le terrain, une version Web responsive (PWA) lui permet de gérer son profil administratif, de consulter ses factures et de demander ses virements depuis un ordinateur.

4.4 Flux de données en Temps Réel (Séquence UI)

L'utilisation des WebSockets permet de mettre à jour l'interface sans action de l'utilisateur. Le diagramme suivant illustre la séquence de mise à jour "Push" lorsqu'un livreur accepte une course.

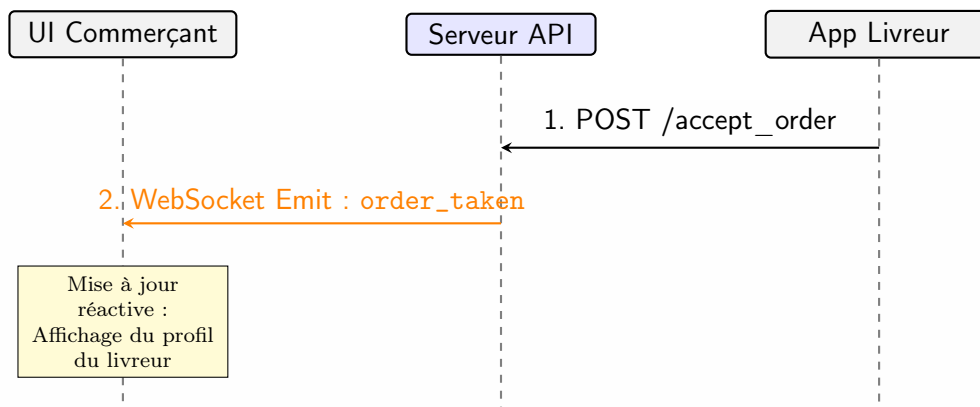


FIGURE 4 – Séquence de mise à jour asynchrone de l’interface via WebSockets.

4.5 Directives Techniques : Langages et Frameworks (Stack Front-End)

Afin d’assurer une base de code homogène, maintenable et performante, l’équipe de développement devra maîtriser et appliquer rigoureusement la stack technologique suivante pour toute l’interface Web :

Cœur de l’Application (Framework & Langage)

- **Next.js (Framework basé sur React)** : Il s’agit du moteur principal de l’application Web. Les développeurs devront utiliser le nouveau paradigme *App Router*. Il est crucial pour l’équipe de comprendre la distinction entre les *Server Components* (utilisés pour le rendu rapide et le SEO des pages publiques) et les *Client Components* (réservés aux dashboards interactifs nécessitant des *Hooks* React comme les tableaux de bord commerçant et admin).
- **TypeScript (Langage)** : L’intégralité du projet front-end sera codée en TypeScript. Les développeurs ont pour directive d’utiliser un typage strict (`strict: true` dans la configuration). Ils devront définir des *Interfaces* et des *Types* clairs pour les entités métier (Commande, Livreur, StatutHandshake) afin d’éliminer les erreurs d’exécution liées aux données de l’API.

Intégration Graphique et Composants (UI)

- **Tailwind CSS (Styling)** : Les développeurs utiliseront l’approche utilitaire de Tailwind directement dans le code JSX/TSX. Cela garantit un design 100% responsive (mobile-first) et évite le code CSS mort.
- **Shadcn/UI & Radix UI (Composants de base)** : Plutôt que de coder les éléments interactifs complexes de zéro (modales de validation de codes, menus dé-

roulants, popovers), les développeurs utiliseront ces bibliothèques. Cela garantit un comportement natif parfait et le respect strict des normes d'accessibilité (ARIA).

Gestion des Données, Formulaires et Temps Réel

- **TanStack Query / React Query (Data Fetching)** : Pour toutes les requêtes REST standards (historique, récupération de profil). React Query gérera le cache, la re-validation en arrière-plan et fournira nativement les états de chargement (`isLoading`) et d'erreur.
- **React Hook Form + Zod (Formulaires)** : La création de compte (Commerçant/Livreur) et la soumission du KYC nécessitent des formulaires robustes. Zod sera utilisé pour définir le schéma de validation (ex : vérifier qu'un code postal est valide) et React Hook Form gérera les saisies utilisateur sans provoquer de re-rendus inutiles de l'interface.
- **Socket.io-client (WebSockets)** : Pour le suivi GPS et les notifications en direct. Les développeurs devront encapsuler la logique WebSocket dans des *Hooks* personnalisés (ex : `useOrderTracking()`) qui écouteront les événements du serveur et déclencheront les changements d'état locaux dans les *Client Components* de React.

5 Architecture Backend & Conception de l'API

Le serveur (Backend) centralise la logique métier, assure l'intégrité des transactions (ACID) et orchestre les communications en temps réel. Cette section définit les standards technologiques et architecturaux que l'équipe de développement devra scrupuleusement respecter.

5.1 Directives Techniques : La Stack Backend

Pour garantir la cohérence avec le Front-End et assurer une grande maintenabilité, l'écosystème Node.js/TypeScript a été retenu, structuré autour d'un framework d'entreprise.

Cœur du Serveur et Logique Métier

- **NestJS (Framework Node.js)** : L'API sera développée avec NestJS. Son architecture modulaire (basée sur Angular), son injection de dépendances et son utilisation intensive des décorateurs (`@Controller`, `@Injectable`) forcent les développeurs à écrire un code propre (Clean Architecture).
- **TypeScript (Langage)** : Typage strict obligatoire. Les développeurs devront utiliser des *Data Transfer Objects* (DTO) validés par `class-validator` pour s'assurer que les payloads (JSON) reçus du Front-End ou des plugins e-commerce sont parfaitement conformes avant tout traitement.

Persistance et Base de Données

- **PostgreSQL + PostGIS (SGBD)** : Base de données relationnelle principale. L'extension PostGIS est indispensable pour stocker les coordonnées GPS des locaux commerciaux et des livreurs, et permettre des requêtes spatiales efficaces (ex : "Trouver les livreurs dans un rayon de 5km").
- **Prisma (ORM)** : Utilisé pour interagir avec PostgreSQL. Prisma génère un client typé dynamiquement depuis le schéma de base de données, offrant une auto-complétion parfaite et prévenant les injections SQL.

Temps Réel, Cache et Tâches de Fond

- **Redis (In-Memory Data Store)** : Utilisé pour trois rôles cruciaux :
 1. Stocker la position GPS éphémère des livreurs.
 2. Gérer le système de file d'attente (Queues) avec la librairie **BullMQ** pour distribuer les requêtes de livraison (dispatch).

3. Gérer le cache des requêtes API fréquentes.
- **Socket.io (WebSockets)** : Intégré via les *Gateways* de NestJS pour émettre les événements bidirectionnels (tracking, notifications de nouvelle course) avec une faible latence.

5.2 Diagramme d'Architecture et Flux de Données

Le diagramme ci-dessous illustre comment les différents composants du Backend interagissent pour traiter une requête de création de commande depuis un plugin E-commerce, jusqu'à la notification des livreurs.

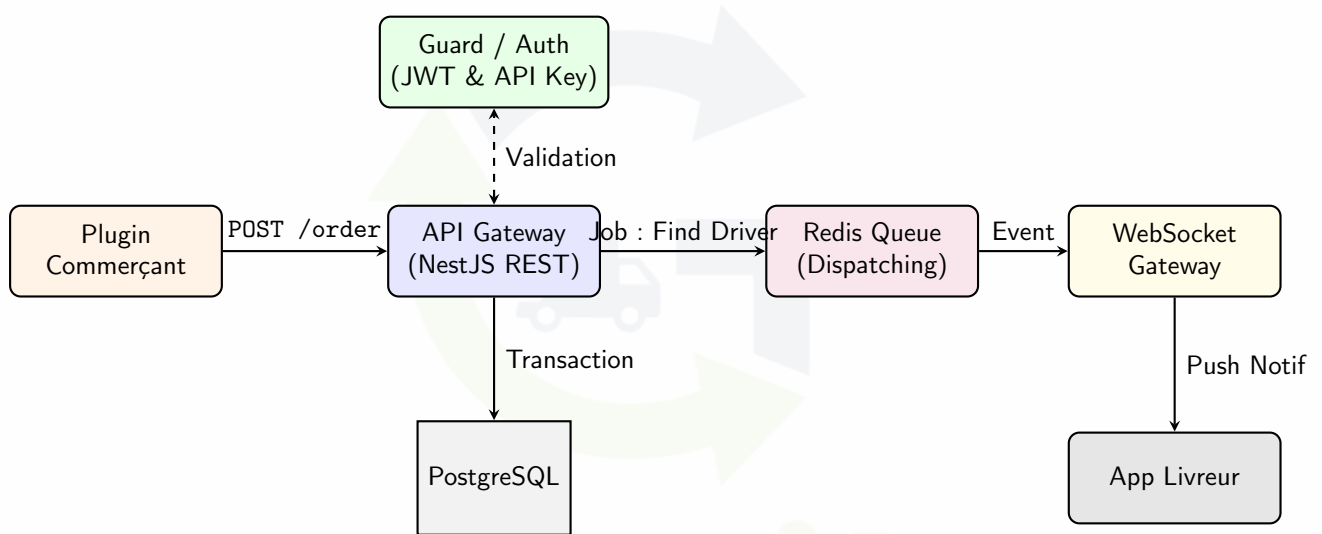


FIGURE 5 – Architecture des composants Backend (Vue orthogonale sans chevauchement).

5.3 Modélisation de la Base de Données (MCD Simplifié)

Le schéma relationnel doit être strict. L'équipe Backend utilisera le concept de transactions ACID pour toute modification impliquant les codes de sécurité ou le solde financier. Voici les entités principales de notre domaine :

5.4 Directives Opérationnelles pour les Développeurs Backend

- **Sécurité des "Handshakes"** : Les codes A (ramassage) et B (livraison) générés par le Backend ne doivent jamais transiter en clair dans les logs.
- **Gestion des accès concurrents (Race Conditions)** : Lors de l'acceptation d'une course par un livreur, le Backend doit utiliser un verrouillage de ligne de base de données (`SELECT ... FOR UPDATE` via Prisma) ou un mutex Redis pour garantir que deux livreurs ne peuvent pas s'assigner la même commande simultanément.

- **RBAC (Role-Based Access Control)** : Utiliser les *Guards* de NestJS pour interdire l'accès à certaines routes. Par exemple, seul le rôle **ADMIN** peut accéder à la route `PUT /livreurs/:id/kyc-approve`.



6 Extension WooCommerce (Plugin Commerçant)

L'extension WooCommerce devra permettre de connecter une boutique WordPress/WooCommerce à l'API de livraison GoMile. Elle aura pour rôle d'ajouter une méthode de livraison dédiée, de calculer un devis de livraison, de transmettre les commandes éligibles à l'API GoMile et de recevoir les mises à jour de statut envoyées par la plateforme.

La documentation technique détaillée de l'extension sera fournie dans le document **guide-plugin.pdf**. Ce guide décrira l'architecture interne du plugin, les hooks WordPress/WooCommerce utilisés, les flux API, les métadonnées de commande et les diagrammes de séquence.

6.1 Fonctionnalités attendues

L'extension devra couvrir les besoins suivants :

- **Méthode de livraison WooCommerce** : ajouter une méthode **Gomile Delivery** configurable dans les zones de livraison WooCommerce et sélectionnable par le client au panier ou au checkout.
- **Calcul dynamique du tarif** : demander un devis à l'API GoMile, notamment à partir de l'adresse d'enlèvement, de l'adresse de livraison et du poids estimé du panier. Le résultat devra pouvoir être mis en cache temporairement afin d'éviter des appels API répétés.
- **Création automatique de livraison** : après validation d'une commande WooCommerce utilisant la méthode GoMile, transmettre à l'API les informations nécessaires à la création d'une livraison : nom du client, téléphone, adresse de livraison, poids et référence de commande WooCommerce.
- **Suivi dans WooCommerce** : stocker les informations liées à la livraison dans les métadonnées de la commande : statut GoMile, identifiant de livraison, URL de suivi, dernière erreur API et date de dernière mise à jour.
- **Annulation de livraison** : déclencher une demande d'annulation auprès de l'API GoMile lorsqu'une commande WooCommerce est annulée.
- **Webhooks GoMile** : exposer une route REST WordPress permettant à l'API GoMile de notifier les changements de statut de livraison et, le cas échéant, de mettre à jour le statut WooCommerce de la commande.

6.2 Contraintes techniques

L'extension devra respecter les standards de développement WordPress et WooCommerce afin de rester compatible avec l'administration, les zones de livraison et les flux de

commande existants.

- **Langage** : l'extension sera développée en PHP, dans le respect des conventions WordPress. La logique devra être structurée de manière à limiter les conflits de nommage dans l'environnement global de WordPress.
- **Intégration WooCommerce** : l'extension devra s'appuyer sur les hooks WooCommerce pour déclarer la méthode de livraison, calculer les tarifs, traiter les commandes et réagir aux annulations.
- **Communication API** : les échanges avec l'API GoMile devront utiliser l'API HTTP WordPress, par exemple `wp_remote_request`, afin de bénéficier de la gestion native des délais d'expiration, des certificats SSL et des erreurs réseau.
- **Configuration administrateur** : une page de réglages devra permettre de configurer l'URL de l'API, la clé API, les endpoints utilisés, le secret webhook, le délai d'expiration HTTP et l'adresse d'enlèvement.
- **Sécurité** : les webhooks devront être protégés par une signature HMAC-SHA256 transmise dans un en-tête HTTP. Le plugin devra vérifier cette signature avant d'accepter une mise à jour de statut.
- **Robustesse** : l'extension devra éviter les créations de livraison en double, notamment en stockant l'état de synchronisation dans les métadonnées de la commande WooCommerce.

6.3 Flux principal attendu

Le flux principal devra être le suivant :

1. Le client sélectionne la méthode de livraison **Gomile Delivery** au panier ou au checkout.
2. WooCommerce demande le calcul du tarif de livraison.
3. L'extension construit une demande de devis et interroge l'API GoMile.
4. Le tarif retourné est affiché dans WooCommerce.
5. Après validation de la commande, l'extension vérifie que la méthode de livraison utilisée est bien GoMile.
6. L'extension transmet la commande à l'API GoMile pour créer la livraison.
7. Les informations retournées par l'API sont stockées dans les métadonnées de la commande WooCommerce.
8. Les mises à jour ultérieures sont reçues par webhook et répercutées dans WooCommerce.

6.4 Livrables attendus

Les livrables liés à cette extension sont les suivants :

- le code source du plugin WooCommerce ;
- une page de configuration administrateur ;
- les appels API nécessaires au devis, à la création de livraison, à l'annulation et au suivi ;
- les tests unitaires couvrant les principaux comportements ;
- une documentation technique complète fournie dans **guide-plugin.pdf**.



7 Application Mobile Livreur

L'application mobile est l'outil de travail exclusif du livreur. Elle doit être robuste, économe en énergie, et capable de maintenir une connexion constante avec le Backend, même dans des conditions réseau dégradées.

7.1 Directives Techniques : La Stack Mobile

Pour maximiser le partage de code et de compétences avec l'équipe Web, le développement s'orientera vers les technologies cross-platform, tout en utilisant des modules natifs pour les fonctionnalités critiques.

- **Framework (UI & Logique) : React Native (avec Expo Bare Workflow).** Ces frameworks permettent de compiler une application native pour iOS et Android à partir d'une seule base de code.
- **Gestion d'État Local : Zustand** (pour React Native). Essentiel pour gérer l'état de la course en cours et le solde du Wallet sans ralentir l'interface.
- **Temps Réel & Notifications : Firebase Cloud Messaging (FCM)** pour les notifications Push (réveil de l'application), couplé à **Socket.io-client** pour le flux de données continu lorsque l'application est ouverte.
- **Géolocalisation métier** : Utilisation de bibliothèques natives spécialisées (ex : **react-native-background-geolocation**) pour garantir le suivi GPS même quand le téléphone est verrouillé, tout en optimisant la batterie.

7.2 Diagramme d'États : Cycle d'Activité et Tracking GPS

Le comportement du composant GPS et les requêtes réseau dépendent entièrement du statut du livreur. Le diagramme ci-dessous dicte aux développeurs quand activer ou désactiver l'envoi de la position pour préserver la batterie.

7.3 Directives d'Implémentation par Fonctionnalité

L'équipe de développement mobile devront porter une attention particulière aux points d'intégration suivants :

1. Tracking GPS (L'enjeu critique)

- **Throttling et Batching** : L'application ne doit pas envoyer une requête HTTP à chaque mètre parcouru. Elle doit accumuler les points GPS locaux (SQ-Lite/AsyncStorage) et les envoyer par lots (batch) au Backend via WebSocket toutes les 5 à 10 secondes.

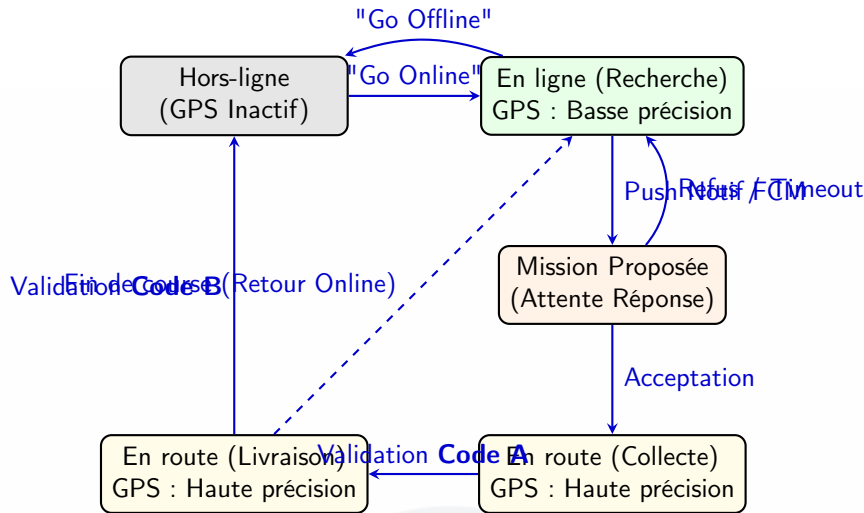


FIGURE 6 – Machine à états du Livreur dictant le comportement du module GPS.

- **Permissions OS** : Le code doit gérer rigoureusement les demandes de permissions "Localiser toujours" (iOS/Android). Sans cette permission, le système d'exploitation tuera l'application en arrière-plan.

2. Gestion des Missions et Push Notifications

- **Deep Linking** : Lorsqu'une notification "Nouvelle Course" arrive, un appui sur celle-ci doit ouvrir l'application directement sur la modale de la mission (Deep Link interne).
- **Mécanisme de Timeout** : La proposition de course doit inclure un compte à rebours visuel (ex : 30 secondes). Si le livreur ne répond pas, l'application doit rejeter silencieusement la course pour que le Backend la réaffecte à un autre livreur (Redis Queue).

3. Navigation et Itinéraires

- L'application mobile ne doit pas calculer les itinéraires elle-même. Les développeurs implémenteront des liens profonds (URI Schemes) pour déléguer la navigation aux applications natives du téléphone.
- **Exemple d'implémentation** : L'appui sur le bouton "Naviguer" déclenchera un `Linking.openURL('waze://?ll=lat,lng&navigate=yes')` pour Waze, ou `google.navigation:q=lat,lng` pour Google Maps.

4. Module Wallet et Virements

- **Sécurité Financière** : Le solde affiché dans l'application doit être purement consultatif (lecture seule). La demande de virement appelle un endpoint Backend strict (ex : intégré avec l'API **Stripe Connect** ou équivalent) qui vérifiera le solde réel côté serveur avant d'initier le transfert bancaire.



8 Conclusion

Ce projet offre une solution "clé en main" pour la logistique urbaine. La modularité entre le plugin WooCommerce, l'API centrale et l'application mobile permet une scalabilité horizontale vers d'autres plateformes (Shopify, PrestaShop) à l'avenir.

